

Kotlin Fundamentals

A colorful pocket guide with runnable examples

📦 Variables
🛡️ Null Safety
⚙️ Functions
🧺 Collections
🏗️ Classes
⚡ Coroutines
🚀 Mini Project

12 chapters · 55+ code samples with output · exercises & cheat sheet

PRODUCED BY



Jin.EX

Kotlin Fundamentals

A Jin.EX production

Contents

01 👋 Hello, Kotlin! 3

02 📦 Variables & Types 5

03 🛡️ Null Safety 8

04 🔀 Control Flow 11

05 ⚙️ Functions 15

06 🧺 Collections 19

07 🏗️ Classes & Objects 23

08 🧬 Inheritance & Interfaces 27

09 🔭 Scope Functions 30

10 🧯 Exceptions & Generics 33

11 ⚡ Coroutines Intro 35

12 🚀 Mini Project 37

★ ⚡ Cheat Sheet 39

✓ ✅ Exercise Solutions 40

CHAPTER 1

Hello, Kotlin!

Your first program and the basics of syntax



Kotlin is a modern, concise and safe programming language made by JetBrains. It runs on the JVM, powers most new **Android** apps, and also works for servers, desktop and even the web.

Your first program

Every Kotlin program starts in a function called `main`.

Main.kt

```
fun main() {
    println("Hello, Kotlin!")
}
```

▶ Output

```
Hello, Kotlin!
```

- `fun` declares a function.
- `println` prints a line of text.
- No semicolons needed at the end of lines.

Comments

Comments.kt

```kotlin
// A single-line comment

/* A multi-line
   comment */

/**
 * KDoc: documentation comment
 */
```

print vs println

Main.kt

```kotlin
fun main() {
    print("Hello, ")
    print("world")
    println("!")
    println("New line")
}
```

▶ Output

```
Hello, world!
New line
```

> 💡 **Tip**
> Run Kotlin instantly in your browser at **play.kotlinlang.org** — no install needed. On a computer, use IntelliJ IDEA or Android Studio.

CHAPTER 2

Variables & Types

val, var, basic types and string templates



val vs var

Kotlin has two kinds of variables:

Keyword	Meaning	Example
`val`	Read-only (cannot be reassigned)	val name = "Ann"
`var`	Mutable (can change)	var age = 20

Main.kt

```
fun main() {
    val name = "Ann"
    var age = 20
    age = 21          // OK
    // name = "Bob"   // Error! val
    println("$name is $age")
}
```

▶ Output

```
Ann is 21
```

📌 **Remember**

Prefer `val` by default. Use `var` only when the value really needs to change.

Basic types

Type	Literal	Use
Int	42	Whole numbers
Long	42L	Big whole numbers
Double	3.14	Decimals
Float	3.14f	Smaller decimals
Boolean	true	true / false
Char	'A'	One character
String	"Hi"	Text

Type inference

Kotlin figures out the type for you, but you can also write it explicitly:

Types.kt

```
val a = 10           // Int (inferred)
val b: Double = 2.5  // explicit type
val c: Long = 100L
val isOk: Boolean = true
```

String templates

Put variables inside strings with `$`, and expressions with `${ }`.

Main.kt

```kotlin
fun main() {
    val x = 7
    val y = 3
    println("x = $x")
    println("sum = ${x + y}")
    val word = "Kotlin"
    println("${word.length} letters")
}
```

▶ Output

```
x = 7
sum = 10
6 letters
```

Type conversion

Main.kt

```kotlin
fun main() {
    val n = 5
    val d: Double = n.toDouble()
    val s = "123".toInt()
    println(d + s)
}
```

▶ Output

```
128.0
```

⚠️ **Watch out**

Kotlin never converts number types automatically. `val d: Double = 5` is an error — use `5.0` or `.toDouble()`.

✍️ **Try it yourself**

Create `val city` and `var temperature`. Change the temperature and print: *"In Yerevan it is 25°C"* using a string template.

CHAPTER 3

Null Safety

Kotlin's superpower against crashes



In many languages, using a *null* value causes the famous **NullPointerException**. Kotlin prevents this at compile time.

Nullable types with ?

Nullable.kt

```kt
var a: String = "hi"
// a = null      // Error!

var b: String? = "hi"
b = null         // OK, b is nullable
```

The safe-call operator ?.

Calls the member only if the value is not null, otherwise returns null.

Main.kt

```kt
fun main() {
    val name: String? = null
    println(name?.length)
    val city: String? = "Paris"
    println(city?.length)
}
```

▶ Output

```
null
5
```

The Elvis operator ?:

Provides a default value when the left side is null.

Main.kt

```
fun main() {
    val input: String? = null
    val len = input?.length ?: 0
    println("Length: $len")
}
```

▶ Output

```
Length: 0
```

let — run code only if not null

Main.kt

```
fun main() {
    val email: String? = "me@mail.com"
    email?.let {
        println("Sending to $it")
    }
}
```

▶ Output

```
Sending to me@mail.com
```

Smart casts

SmartCast.kt

```
fun printLen(s: String?) {
    if (s != null) {
        // s is now treated as String
        println(s.length)
    }
}
```

> ⚠️ **Watch out**
> `!!` (not-null assertion) says “trust me, it's not null”. If it is null, the app crashes. Avoid it whenever you can.

Danger.kt

```
val x: String? = null
println(x!!.length) // 💥 NullPointerException
```

CHAPTER 4

Control Flow

if, when, ranges and loops



if is an expression

In Kotlin, `if` returns a value — so there's no need for a ternary operator.

Main.kt

```
fun main() {
    val a = 8
    val b = 5
    val max = if (a > b) a else b
    println("Max: $max")
}
```

▶ Output

```
Max: 8
```

when — a powerful switch

Main.kt

```
fun describe(x: Any): String = when (x) {
    1          -> "One"
    "Hello"    -> "Greeting"
    is Long    -> "A Long number"
    !is String -> "Not a string"
    else       -> "Unknown"
}

fun main() {
    println(describe(1))
    println(describe("Hello"))
    println(describe(3.5))
}
```

▶ Output

```
One
Greeting
Not a string
```

Grade.kt

```
fun grade(score: Int) = when (score) {
    in 90..100 -> "A"
    in 75..89  -> "B"
    in 60..74  -> "C"
    else       -> "F"
}
```

Ranges

Range	Values
`1..5`	1, 2, 3, 4, 5
`1 until 5`	1, 2, 3, 4
`1..<5`	1, 2, 3, 4
`5 downTo 1`	5, 4, 3, 2, 1
`1..10 step 3`	1, 4, 7, 10

for loops

Main.kt

```
fun main() {
    for (i in 1..3) print("$i ")
    println()
    val fruits = listOf("🍎", "🍌", "🍇")
    for (f in fruits) print(f)
    println()
    for ((i, f) in fruits.withIndex())
        println("$i: $f")
}
```

▶ Output

```
1 2 3
🍎🍌🍇
0: 🍎
1: 🍌
2: 🍇
```

while and do-while

Main.kt

```
fun main() {
    var n = 3
    while (n > 0) {
        println("n = $n")
        n--
    }
    do {
        println("Runs at least once")
    } while (false)
}
```

▶ Output

```
n = 3
n = 2
n = 1
Runs at least once
```

break & continue

Main.kt

```
fun main() {
    for (i in 1..10) {
        if (i % 2 == 0) continue
        if (i > 7) break
        print("$i ")
    }
}
```

▶ Output

```
1 3 5 7
```

✍️ **Try it yourself**

Write a FizzBuzz: for 1..15 print “Fizz” for multiples of 3, “Buzz” for 5, “FizzBuzz” for both, else the number. Hint: use `when` with no argument.

CHAPTER 5

Functions

Parameters, lambdas and extensions



Declaring functions

Main.kt

```
fun add(a: Int, b: Int): Int {
    return a + b
}

// Single-expression function
fun multiply(a: Int, b: Int) = a * b

fun main() {
    println(add(2, 3))
    println(multiply(4, 5))
}
```

▶ Output

```
5
20
```

Default & named arguments

Main.kt

```kotlin
fun greet(name: String, greeting: String = "Hello") {
    println("$greeting, $name!")
}

fun main() {
    greet("Ann")
    greet("Bob", "Hi")
    greet(greeting = "Hey", name = "Sam")
}
```

▶ Output

```
Hello, Ann!
Hi, Bob!
Hey, Sam!
```

Unit = no return value

Functions that return nothing useful return `Unit` (like *void*). You can omit it.

vararg

Main.kt

```kotlin
fun sumAll(vararg nums: Int): Int = nums.sum()

fun main() {
    println(sumAll(1, 2, 3, 4))
}
```

▶ Output

```
10
```

Lambdas

A lambda is a function without a name, written in curly braces.

Main.kt

```
fun main() {
    val square = { x: Int -> x * x }
    println(square(6))

    val hello: () -> Unit = { println("Hi!") }
    hello()
}
```

▶ Output

```
36
Hi!
```

Higher-order functions

Functions that take other functions as parameters.

Main.kt

```
fun operate(a: Int, b: Int,
            op: (Int, Int) -> Int): Int {
    return op(a, b)
}

fun main() {
    println(operate(3, 4) { x, y -> x + y })
    println(operate(3, 4) { x, y -> x * y })
}
```

▶ Output

```
7
12
```

> 💡 **Tip**
> If the last parameter is a function, you can put the lambda **outside** the parentheses — this is called a *trailing lambda*.

Extension functions

Add new functions to existing classes without changing them!

Main.kt

```
fun String.shout() = uppercase() + "!!!"

fun Int.isEven() = this % 2 == 0

fun main() {
    println("kotlin".shout())
    println(4.isEven())
}
```

▶ Output

```
KOTLIN!!!
true
```

CHAPTER 6

Collections

Lists, sets, maps and their magic functions



Read-only vs mutable

Type	Read-only	Mutable
List	listOf()	mutableListOf()
Set	setOf()	mutableSetOf()
Map	mapOf()	mutableMapOf()

Lists

Main.kt

```
fun main() {
    val nums = listOf(1, 2, 3)
    println(nums[0])
    println(nums.size)

    val todo = mutableListOf("Code")
    todo.add("Sleep")
    todo.remove("Code")
    println(todo)
}
```

▶ Output

```
1
3
[Sleep]
```

Sets — no duplicates

Main.kt

```
fun main() {
    val tags = setOf("a", "b", "a", "c")
    println(tags)
    println("b" in tags)
}
```

▶ Output

```
[a, b, c]
true
```

Maps — key/value pairs

Main.kt

```
fun main() {
    val ages = mutableMapOf(
        "Ann" to 21,
        "Bob" to 30
    )
    ages["Sam"] = 25
    println(ages["Bob"])
    for ((name, age) in ages)
        println("$name → $age")
}
```

▶ Output

```
30
Ann → 21
Bob → 30
Sam → 25
```

Powerful operations

Main.kt

```kotlin
fun main() {
    val n = listOf(1, 2, 3, 4, 5, 6)

    println(n.filter { it % 2 == 0 })
    println(n.map { it * 10 })
    println(n.sum())
    println(n.first { it > 3 })
    println(n.any { it > 5 })
    println(n.count { it < 3 })
    println(n.sortedDescending())
}
```

▶ Output

```
[2, 4, 6]
[10, 20, 30, 40, 50, 60]
21
4
true
2
[6, 5, 4, 3, 2, 1]
```

📌 **Remember**

`it` is the default name for a lambda's single parameter.

Chaining

Main.kt

```
data class User(val name: String, val age: Int)

fun main() {
    val users = listOf(
        User("Ann", 17), User("Bob", 25),
        User("Eve", 31), User("Tom", 15)
    )
    val adults = users
        .filter { it.age >= 18 }
        .sortedBy { it.name }
        .map { it.name }
    println(adults)
    println(users.groupBy { it.age >= 18 }
        .mapValues { it.value.size })
}
```

▶ Output

```
[Bob, Eve]
{false=2, true=2}
```

✍️ **Try it yourself**

Given `listOf("apple", "kiwi", "banana", "fig")`, print only words longer than 3 letters, in uppercase.

CHAPTER 7

Classes & Objects

Constructors, properties and data classes



A simple class

Main.kt

```
class Person(val name: String, var age: Int) {
    fun introduce() {
        println("I'm $name, $age years old")
    }
}

fun main() {
    val p = Person("Ann", 21)
    p.introduce()
    p.age++
    println(p.age)
}
```

▶ Output

```
I'm Ann, 21 years old
22
```

📌 **Remember**

Kotlin has no `new` keyword. Just call the class like a function.

init block & default values

Main.kt

```
class Car(val brand: String,
          val speed: Int = 0) {
    init {
        println("Created a $brand")
    }
}

fun main() {
    val c = Car("Tesla")
    println(c.speed)
}
```

▶ Output

```
Created a Tesla
0
```

Custom getters & setters

Properties.kt

```
class Rect(val w: Int, val h: Int) {
    val area: Int
        get() = w * h
}

class Counter {
    var count = 0
        set(value) {
            if (value >= 0) field = value
        }
}
```

Data classes

Perfect for holding data. You get `toString`, `equals`, `hashCode` and `copy` for free.

Main.kt

```kotlin
data class Point(val x: Int, val y: Int)

fun main() {
    val p1 = Point(1, 2)
    val p2 = Point(1, 2)
    println(p1)
    println(p1 == p2)
    val p3 = p1.copy(y = 9)
    println(p3)
    val (x, y) = p3
    println("x=$x y=$y")
}
```

▶ Output

```
Point(x=1, y=2)
true
Point(x=1, y=9)
x=1 y=9
```

object — a singleton

Main.kt

```kotlin
object AppConfig {
    const val VERSION = "1.0"
    fun info() = "App v$VERSION"
}

fun main() = println(AppConfig.info())
```

▶ Output

```
App v1.0
```

companion object

Main.kt

```
class User private constructor(val name: String) {
    companion object {
        fun create(name: String) = User(name)
    }
}

fun main() {
    val u = User.create("Ann")
    println(u.name)
}
```

▶ Output

```
Ann
```

Enum classes

Main.kt

```
enum class Color(val hex: String) {
    RED("#FF0000"), GREEN("#00FF00")
}

fun main() {
    println(Color.RED.hex)
    for (c in Color.entries) print("$c ")
}
```

▶ Output

```
#FF0000
RED GREEN
```

CHAPTER 8

Inheritance & Interfaces

open, abstract, interfaces and sealed classes



open classes

Classes are **final** by default. Mark them `open` to allow inheritance.

Main.kt

```
open class Animal(val name: String) {
    open fun sound() = "..."
}

class Dog(name: String) : Animal(name) {
    override fun sound() = "Woof!"
}

class Cat(name: String) : Animal(name) {
    override fun sound() = "Meow!"
}

fun main() {
    val pets = listOf(Dog("Rex"), Cat("Tom"))
    for (p in pets)
        println("${p.name}: ${p.sound()}")
}
```

▶ Output

```
Rex: Woof!
Tom: Meow!
```

Abstract classes

Main.kt

```
abstract class Shape {
    abstract fun area(): Double
    fun describe() = "Area: ${area()}"
}

class Square(val side: Double) : Shape() {
    override fun area() = side * side
}

fun main() = println(Square(3.0).describe())
```

▶ Output

```
Area: 9.0
```

Interfaces

Main.kt

```
interface Clickable {
    fun click()
    fun showOff() = println("I'm clickable")
}

class Button : Clickable {
    override fun click() = println("Clicked!")
}

fun main() {
    val b = Button()
    b.click()
    b.showOff()
}
```

▶ Output

```
Clicked!
I'm clickable
```

> 💡 **Tip**
> A class can extend only **one** class but implement **many** interfaces.

Sealed classes

A closed set of subclasses — `when` knows all cases, so no `else` is needed.

Main.kt

```
sealed class Result {
    data class Success(val data: String) : Result()
    data class Error(val msg: String) : Result()
    object Loading : Result()
}

fun show(r: Result) = when (r) {
    is Result.Success -> "✅ ${r.data}"
    is Result.Error   -> "❌ ${r.msg}"
    Result.Loading    -> "⏳ Loading..."
}

fun main() {
    println(show(Result.Success("Done")))
    println(show(Result.Loading))
}
```

▶ Output

```
✅ Done
⏳ Loading...
```

CHAPTER 9

Scope Functions

let, run, with, apply and also



Scope functions run a block of code on an object. They make code shorter and more readable.

Function	Object as	Returns	Typical use
let	it	Lambda result	Null checks, transforms
run	this	Lambda result	Compute a result
with	this	Lambda result	Group calls on object
apply	this	The object	Configure an object
also	it	The object	Side effects (logging)

apply — configure

Main.kt

```
class Profile {
    var name = ""
    var age = 0
}

fun main() {
    val p = Profile().apply {
        name = "Ann"
        age = 21
    }
    println("${p.name}, ${p.age}")
}
```

▶ Output

```
Ann, 21
```

let — transform / null check

Main.kt

```
fun main() {
    val len = "Kotlin".let { it.length * 2 }
    println(len)
}
```

▶ Output

```
12
```

also — side effects

Main.kt

```
fun main() {
    val list = mutableListOf(1, 2)
        .also { println("Before: $it") }
    list.add(3)
    println("After: $list")
}
```

▶ Output

```
Before: [1, 2]
After: [1, 2, 3]
```

with & run

Main.kt

```
fun main() {
    val sb = StringBuilder()
    val text = with(sb) {
        append("Hello ")
        append("World")
        toString()
    }
    println(text)

    val r = "abc".run { uppercase() + length }
    println(r)
}
```

▶ Output

```
Hello World
ABC3
```

CHAPTER 10

Exceptions & Generics

Handling errors and writing reusable code



try / catch / finally

Main.kt

```
fun main() {
    try {
        val n = "abc".toInt()
        println(n)
    } catch (e: NumberFormatException) {
        println("Not a number!")
    } finally {
        println("Done")
    }
}
```

▶ Output

```
Not a number!
Done
```

try is an expression

Main.kt

```
fun main() {
    val n = try { "42".toInt() }
            catch (e: Exception) { 0 }
    println(n)
}
```

▶ Output

```
42
```

Throwing & checks

Bank.kt

```kotlin
fun withdraw(balance: Int, amount: Int): Int {
    require(amount > 0) { "Amount must be > 0" }
    if (amount > balance)
        throw IllegalStateException("No money")
    return balance - amount
}
```

> 💡 **Tip**
> Safe conversion: `"abc".toIntOrNull()` returns `null` instead of throwing.

Generics

Write code that works with any type.

Main.kt

```kotlin
class Box<T>(val item: T) {
    fun get(): T = item
}

fun <T> firstOrDefault(list: List<T>, d: T): T =
    if (list.isEmpty()) d else list[0]

fun main() {
    println(Box("Gift").get())
    println(Box(42).get())
    println(firstOrDefault(listOf<Int>(), -1))
}
```

▶ **Output**

```
Gift
42
-1
```

CHAPTER 11

Coroutines Intro

A first look at async code



Coroutines let you write asynchronous code (network calls, timers) that looks like normal sequential code. They are very light — you can run thousands of them.

> ⚠️ **Watch out**
> Coroutines need the **kotlinx-coroutines-core** library. Android projects usually include it already.

launch & delay

Main.kt

```
import kotlinx.coroutines.*

fun main() = runBlocking {
    launch {
        delay(1000L)
        println("World!")
    }
    println("Hello,")
}
```

▶ Output

```
Hello,
World!
```

suspend functions

Main.kt

```
import kotlinx.coroutines.*

suspend fun fetchUser(): String {
    delay(500)  // pretend network call
    return "Ann"
}

fun main() = runBlocking {
    val user = fetchUser()
    println("Loaded $user")
}
```

▶ Output

```
Loaded Ann
```

async — run in parallel

Main.kt

```
import kotlinx.coroutines.*

suspend fun getA(): Int { delay(1000); return 10 }
suspend fun getB(): Int { delay(1000); return 20 }

fun main() = runBlocking {
    val a = async { getA() }
    val b = async { getB() }
    println("Sum: ${a.await() + b.await()}")
}   // takes ~1 second, not 2!
```

▶ Output

```
Sum: 30
```

📌 **Remember**

`suspend` functions can only be called from a coroutine or another suspend function.

CHAPTER 12

Mini Project

Put it all together: a To-Do app



Let's combine data classes, enums, collections, null safety and extension functions in one small program.

TodoApp.kt

```
enum class Priority { LOW, MEDIUM, HIGH }

data class Task(
    val id: Int,
    val title: String,
    val priority: Priority = Priority.MEDIUM,
    var done: Boolean = false
)

class TodoList {
    private val tasks = mutableListOf<Task>()
    private var nextId = 1

    fun add(title: String, p: Priority =
Priority.MEDIUM) =
        Task(nextId++, title, p).also { tasks += it }

    fun complete(id: Int) {
        tasks.find { it.id == id }?.done = true
    }

    fun pending() = tasks.filter { !it.done }
        .sortedByDescending { it.priority }

    fun progress(): Int =
        if (tasks.isEmpty()) 0
        else tasks.count { it.done } * 100 /
tasks.size
}

fun Task.pretty(): String {
    val box = if (done) "☑" else "☐"
```

```
    return "$box #$id $title [$priority]"
}

fun main() {
    val todo = TodoList()
    todo.add("Learn Kotlin", Priority.HIGH)
    todo.add("Buy milk", Priority.LOW)
    todo.add("Write app")
    todo.complete(2)

    todo.pending().forEach { println(it.pretty()) }
    println("Progress: ${todo.progress()}%")
}
```

▶ **Output**

```
□ #1 Learn Kotlin [HIGH]
□ #3 Write app [MEDIUM]
Progress: 33%
```

✍️ **Try it yourself**

Add a `remove(id)` function, and a `search(text)` that returns tasks whose title contains the text (ignore case).

⚡ Cheat Sheet

Everything on one page

Concept	Syntax
Variable	`val x = 1` / `var y = 2`
Nullable	`String?`
Safe call	`a?.b`
Elvis	`a ?: default`
Template	`"Hi $name ${a+b}"`
Function	`fun f(x: Int) = x * 2`
Lambda	`{ x: Int -> x * 2 }`
Range	`1..10`, `0 until n`
When	`when (x) { 1 -> ... else -> ... }`
List	`listOf(1,2)` / `mutableListOf()`
Map	`mapOf("a" to 1)`
Filter / Map	`list.filter{}.map{}`
Class	`class P(val n: String)`
Data class	`data class P(val n: String)`
Singleton	`object Config`
Inherit	`open class A`; `class B : A()`
Extension	`fun String.x() = ...`
Type check	`if (x is String)`
Cast	`x as String`, `x as? String`
Try	`val v = try {..} catch(e: Exception) {..}`

ANSWERS

Exercise Solutions



Check your work

Variables (Ch. 2)

Main.kt

```
fun main() {
    val city = "Yerevan"
    var temperature = 20
    temperature = 25
    println("In $city it is $temperature°C")
}
```

▶ Output

```
In Yerevan it is 25°C
```

FizzBuzz (Ch. 4)

Main.kt

```
fun main() {
    for (i in 1..15) {
        val s = when {
            i % 15 == 0 -> "FizzBuzz"
            i % 3 == 0  -> "Fizz"
            i % 5 == 0  -> "Buzz"
            else        -> i.toString()
        }
        println(s)
    }
}
```

Collections (Ch. 6)

Main.kt

```kotlin
fun main() {
    val words = listOf("apple", "kiwi",
                       "banana", "fig")
    println(words.filter { it.length > 3 }
                 .map { it.uppercase() })
}
```

▶ Output

```
[APPLE, KIWI, BANANA]
```

To-Do extras (Mini Project)

TodoList additions

```kotlin
fun remove(id: Int) =
    tasks.removeIf { it.id == id }

fun search(text: String) = tasks.filter {
    it.title.contains(text, ignoreCase = true)
}
```

🎉 Congratulations! You now know the fundamentals of Kotlin.
Next steps: build a small Android app with Jetpack Compose, explore coroutines & Flow, and practise on **kotlinlang.org**.